

FORENSICS & STEGANOGRAPHY

CTF Cheat Sheet | File Analysis, Stego, PCAP, Memory, Disk, Audio

STEP 0 — Universal First Steps (Do This For EVERY File)

Before using any specialized tool, run these 4 commands on every file:

```
# 1. Identify real file type (ignore the extension given)
file suspicious_file

# 2. Extract all metadata
exiftool suspicious_file

# 3. Extract all readable strings, grep for flag
strings suspicious_file | grep -i flag
strings suspicious_file | grep -i ctf
strings suspicious_file | grep -i key

# 4. Check for hidden files inside
binwalk suspicious_file
binwalk -e suspicious_file # -e extracts everything found
```

IMPORTANT: The file extension is often **WRONG** or **MISLEADING** in CTF challenges. Always run 'file' first. A .jpg might actually be a ZIP.

TIP: If strings output is huge, pipe it: `strings file | grep -i 'flag|ctf|key|pass|secret'` to filter noise.

1. File Identification & Magic Bytes

Common Magic Bytes (File Signatures)

The first few bytes of a file identify its true type. Use a hex editor to check/fix them.

File Type	Magic Bytes (Hex)	ASCII	Extension
PNG	89 50 4E 47 0D 0A 1A 0A	.PNG....	.png
JPEG	FF D8 FF	...	.jpg / .jpeg
GIF	47 49 46 38	GIF8	.gif
BMP	42 4D	BM	.bmp
ZIP	50 4B 03 04	PK..	.zip / .docx / .jar
PDF	25 50 44 46 2D	%PDF-	.pdf
ELF	7F 45 4C 46	.ELF	Linux binary
PE	4D 5A	MZ	.exe / .dll (Windows)
MP3	49 44 33	ID3	.mp3
WAV	52 49 46 46	RIFF	.wav
GZIP	1F 8B	..	.gz / .tar.gz
7ZIP	37 7A BC AF 27 1C	7z....'.	.7z
RAR	52 61 72 21	Rar!	.rar
SQLite	53 51 4C 69 74 65	SQLite	.db / .sqlite
Class	CA FE BA BE		.class (Java)

Hex Editors — View & Fix Magic Bytes

```
# Terminal hex view
xxd file.bin | head -5      # show first 5 lines
xxd file.bin | grep -i '4950' # search hex pattern
hexdump -C file.bin | head

# Fix corrupted magic bytes (example: fix PNG header)
printf '\x89PNG\r\n\x1a\n' | dd of=broken.png bs=1 seek=0 conv=notrunc

# Online hex editor: hexed.it
# Linux GUI: ghex, bless
# Windows: HxD (free)

# Quick check: is the file what it claims to be?
file image.jpg # if output says 'Zip archive' -> it's actually a ZIP
```

2. Image Steganography

Step-by-Step Image Analysis

```
# Run ALL of these on any suspicious image:

# Step 1: metadata
exiftool image.jpg

# Step 2: strings
strings image.jpg | grep -i flag

# Step 3: hidden files inside
binwalk -e image.jpg

# Step 4: LSB steganography (PNG especially)
zsteg image.png      # tries many LSB combos automatically
zsteg -a image.png   # try all methods

# Step 5: steghide (JPG, BMP — may need password)
steghide extract -sf image.jpg      # try empty password
steghide extract -sf image.jpg -p password
steghide extract -sf image.jpg -p ''

# Step 6: online tools
# stegonline.georgeom.net -> bit plane viewer
# stylesuxx.github.io/steganography -> LSB decode
# aperisolve.com -> runs many tools at once (BEST ONLINE OPTION)
```

TIP: aperisolve.com is the best single online tool — paste your image and it runs binwalk, zsteg, steghide, strings, exiftool all at once.

LSB (Least Significant Bit) Steganography

```
# Data hidden in the lowest bit of each pixel's R, G, or B channel
# Affects image only very slightly — invisible to eye

# Tool: zsteg (best for PNG)
zsteg -a image.png

# Tool: stegsolve (Java GUI) — view each bit plane
java -jar stegsolve.jar
# Open image -> use arrows to cycle through bit planes
# Look for patterns/text in low bit planes

# Online: stegonline.georgeom.net
# -> Browse Bit Planes -> look for hidden text/pattern

# Python manual LSB extract (if you know layout)
```

```

from PIL import Image
img = Image.open('image.png')
pixels = list(img.getdata())
bits = ''
for px in pixels:
    bits += str(px[0] & 1) # LSB of red channel
# Convert bits to text:
msg = ''
for i in range(0, len(bits)-7, 8):
    byte = bits[i:i+8]
    if byte == '00000000': break
    msg += chr(int(byte, 2))
print(msg)

```

Steghide (JPG / BMP)

```

# Extract hidden data (try these passwords in order):
steghide extract -sf image.jpg -p ''
steghide extract -sf image.jpg -p 'password'
steghide extract -sf image.jpg -p 'steghide'
steghide extract -sf image.jpg -p 'secret'
steghide extract -sf image.jpg -p '1234'

# Brute-force steghide password:
stegseek image.jpg rockyou.txt # FAST brute-forcer
stegcracker image.jpg rockyou.txt # alternative

# Check if steghide was used (no password needed for check):
steghide info image.jpg

```

Image Bit Plane & Color Channel Analysis

```

# Stegsolve: view individual color planes, bit planes
# Often reveals hidden text/image in specific bit plane

# Online equivalent: stegonline.georgeom.net
# -> Bit Planes: check R0, G0, B0 (least significant)
# -> Colour Channels: split R, G, B

# Python: extract specific channel
from PIL import Image
img = Image.open('image.png')
r, g, b = img.split()
r.save('red_channel.png')

# Look for differences between similar images:
# XOR two images pixel by pixel
from PIL import Image, ImageChops
img1 = Image.open('a.png')
img2 = Image.open('b.png')
diff = ImageChops.difference(img1, img2)
diff.save('diff.png')

```

Outguess / OpenStego / F5

```

# Outguess (JPG stego tool)
outguess -r image.jpg output.txt

# OpenStego (PNG, with/without password)
openstego extract -sf image.png -p password -xf output.txt

# F5 (JPG, Java-based)

```

```
java Extract image.jpg # extracts if hidden data present
```

3. Audio Steganography

Spectrogram Analysis (Most Common Audio Stego)

```
# Flag often literally VISIBLE as text in the spectrogram

# Audacity (best - free GUI tool):
# 1. Open audio file in Audacity
# 2. Click dropdown arrow on track name
# 3. Select: Spectrogram
# 4. Zoom in and look for text/patterns

# Online spectrogram viewers:
# academo.org/demos/spectrum-analyzer
# spectrogram.online
# dcode.fr/spectral-analysis

# Also check: is there audio only in one channel?
# Audacity -> Tracks -> Stereo to Mono -> check each channel
```

TIP: Spectrogram hiding is extremely common in CTFs. Always view the spectrogram of any audio file — the flag is often written out in the frequency domain.

Audio LSB / Data Hiding

```
# Strings in audio files
strings audio.wav | grep -i flag
strings audio.mp3 | grep -i flag

# Metadata
exiftool audio.mp3
exiftool audio.wav

# Binwalk - hidden files in audio
binwalk -e audio.mp3

# MP3Stego (MP3 specific steganography tool)
mp3stego-decode -X -P password audio.mp3 output.txt

# DeepSound (Windows tool - common in CTFs)
# If steghide doesn't work on audio -> try DeepSound GUI
# Can be run via wine on Linux

# Sonic Visualiser (advanced frequency analysis)
# Layer -> Add Spectrogram -> look for patterns
```

Morse / Other Encodings in Audio

```
# If audio sounds like beeps -> likely Morse code
# Listen carefully: short beep = dot, long beep = dash

# Auto-decode Morse from audio:
# morsecode.world/signal (upload audio file)
# dcode.fr/morse-code (manual decode)

# DTMF (phone dial tones) -> phone number encoded
# multitonedtmf.netlify.app (decode DTMF audio)
# dialabc.com/sound/detect (DTMF detector)

# If audio is reversed -> reverse it in Audacity
# Effect -> Reverse
```

```
# Then listen again / view spectrogram
```

4. File Carving & Hidden Data Extraction

Binwalk — Extract Hidden Files

```
# Scan for embedded files
binwalk file

# Extract everything found
binwalk -e file
binwalk --extract file

# Extract to specific directory
binwalk -e file -C output_dir/

# Scan recursively (extracted files too)
binwalk -Me file

# Common finds: ZIP inside JPG, ELF inside PNG, text files inside MP3

# If binwalk finds a ZIP at offset 12345:
dd if=file.jpg of=extracted.zip bs=1 skip=12345
```

Foremost — File Carving from Disk Images

```
# Carve files from raw disk image / binary blob
foremost -i disk.img -o output_dir/
foremost -t jpg,png,zip,pdf -i disk.img -o out/

# Check output directory for recovered files:
ls -la output_dir/
ls -la output_dir/jpg/
```

Manual Carving with dd

```
# Extract bytes starting at specific offset
dd if=input.bin of=output.zip bs=1 skip=OFFSET
dd if=input.bin of=output.zip bs=1 skip=OFFSET count=SIZE

# Find offsets with:
binwalk file # shows offsets of embedded files
xxd file | grep '504b' # find ZIP magic bytes (PK)

# Example: extract ZIP starting at byte 2048
dd if=suspicious.jpg of=hidden.zip bs=1 skip=2048
unzip hidden.zip
```

ZIP / Archive Issues

```
# Corrupted or password-protected ZIPs
unzip archive.zip
unzip -P password archive.zip

# Crack ZIP password:
fcrackzip -v -u -D -p rockyou.txt archive.zip
john --format=zip hash.txt # after zip2john archive.zip > hash.txt
hashcat -a 0 -m 17200 hash.txt rockyou.txt

# Fix corrupted ZIP:
```

```

zip -FF corrupted.zip --out fixed.zip

# List ZIP contents without extracting:
unzip -l archive.zip

# ZIP comment (flags sometimes here):
python3 -c "import zipfile; z=zipfile.ZipFile('a.zip'); print(z.comment)"

# Password-protected RAR:
unrar x -pPASSWORD archive.rar
john --format=rar5 hash.txt # after rar2john archive.rar > hash.txt

```

5. Network Traffic Analysis (PCAP)

Wireshark — Essential Operations

```

# Open PCAP file:
wireshark capture.pcap

# FIRST: Statistics -> Protocol Hierarchy
# Shows what protocols are present (HTTP, DNS, FTP, etc.)

# Follow a stream (reassemble full conversation):
# Right-click any packet -> Follow -> TCP Stream
# Right-click any packet -> Follow -> HTTP Stream
# Right-click any packet -> Follow -> UDP Stream

# Search for flag string in all packets:
# Edit -> Find Packet -> String -> type 'flag'
# Or: Packet Bytes -> String -> 'CTF{'

# Export all HTTP objects (files transferred):
# File -> Export Objects -> HTTP -> Save All

# Export FTP / SMB / DICOM objects:
# File -> Export Objects -> select protocol

# Filter by protocol:
http      ftp      dns      smtp      telnet
tcp.port == 80      ip.addr == 192.168.1.1
http.request.method == "POST"
ftp-data # actual FTP file transfers

```

tshark — Command Line PCAP Analysis

```

# List all HTTP requests
tshark -r capture.pcap -Y http.request -T fields -e http.host -e http.request.uri

# Extract all strings from PCAP
strings capture.pcap | grep -i flag

# Show all DNS queries
tshark -r capture.pcap -Y dns -T fields -e dns.qry.name

# Extract HTTP data
tshark -r capture.pcap -Y 'http' -T fields -e http.file_data

# Follow TCP stream #0
tshark -r capture.pcap -q -z follow,tcp,ascii,0

# Export specific stream to file

```

```
tshark -r capture.pcap -q -z follow,tcp,raw,0 | xxd -r -p > stream.bin
```

Common PCAP Scenarios in CTFs

```
# Credentials in HTTP POST:
# Filter: http.request.method == POST
# Follow HTTP stream -> look for username= password=

# FTP credentials (sent in cleartext):
# Filter: ftp
# Look for USER and PASS packets

# File transferred via HTTP/FTP:
# File -> Export Objects -> HTTP / FTP

# DNS exfiltration (flag encoded in DNS queries):
# tshark -r file.pcap -Y dns -T fields -e dns.qry.name
# Subdomains often contain base64-encoded data

# Telnet / raw TCP (cleartext):
# Follow TCP stream -> read plaintext commands

# ICMP data (ping with hidden payload):
# Filter: icmp
# Check Data field in each ICMP packet

# TLS/SSL - encrypted traffic:
# If you have the private key: Edit -> Preferences -> RSA Keys
# Or check for SSL key log file hint in challenge

# Wireless (WPA):
# aircrack-ng capture.pcap -w rockyou.txt
# hashcat -a 0 -m 22000 hash.hc22000 rockyou.txt
```

TIP: Always start with Statistics -> Protocol Hierarchy. It tells you exactly what's in the PCAP so you know where to look.

6. Metadata Analysis

Exiftool — Extract All Metadata

```
# Basic usage
exiftool file.jpg
exiftool file.pdf
exiftool file.docx

# Output all fields verbosely
exiftool -v file.jpg

# Show specific field
exiftool -Comment file.jpg
exiftool -GPS* file.jpg # GPS coordinates
exiftool -Author file.pdf

# Fields that often contain flags:
# Comment, Artist, Author, Description, Copyright
# UserComment, XPCComment, Software
# GPS (coordinates -> location clue)
# Thumbnail (embedded thumbnail may differ from main image!)

# Extract embedded thumbnail
exiftool -ThumbnailImage -b file.jpg > thumb.jpg

# Batch all files in directory
exiftool *.jpg | grep -i 'comment\|flag\|ctf'
```

Document Metadata (PDF, DOCX, etc.)

```
# PDF metadata
exiftool document.pdf
pdftools document.pdf
strings document.pdf | grep -i flag

# Hidden text in PDF (white text on white background)
# Open in PDF viewer, Ctrl+A to select all, copy -> paste to see hidden text

# DOCX / Office files (they are ZIPs)
unzip -o document.docx -d docx_extracted/
cat docx_extracted/word/document.xml | grep -i flag
strings docx_extracted/word/document.xml

# Hidden revisions / tracked changes in DOCX
# Open in LibreOffice -> Edit -> Track Changes -> Show Changes

# PNG/JPEG embedded in DOCX
ls docx_extracted/word/media/
```

7. Memory Forensics (Volatility)

Volatility 3 — Essential Commands

```
# Identify OS profile first
vol3 -f memory.dmp windows.info
vol3 -f memory.dmp linux.info

# List running processes
vol3 -f memory.dmp windows.pslist
vol3 -f memory.dmp windows.pstree    # with parent-child tree

# List network connections
vol3 -f memory.dmp windows.netstat

# List open files
vol3 -f memory.dmp windows.filescan
vol3 -f memory.dmp windows.filescan | grep -i flag
vol3 -f memory.dmp windows.filescan | grep -i '.txt\|.jpg\|.zip'

# Dump specific file from memory
vol3 -f memory.dmp windows.dumpfiles --physaddr 0xADDRESS

# Command history
vol3 -f memory.dmp windows.cmdline
vol3 -f memory.dmp windows.cmdscan
vol3 -f memory.dmp windows.consoles    # typed commands

# Registry hives
vol3 -f memory.dmp windows.registry.hivelist
vol3 -f memory.dmp windows.registry.printkey --key 'SOFTWARE\Microsoft\Windows'

# Dump process memory
vol3 -f memory.dmp windows.memmap --pid PID --dump

# Strings in process memory
vol3 -f memory.dmp windows.memmap --pid PID --dump
strings pid.PID.dmp | grep -i flag
```


Volatility 2 (older syntax)

```
# Get profile
volatility -f memory.dmp imageinfo

# Use profile for all subsequent commands
volatility -f memory.dmp --profile=Win7SP1x64 pslist
volatility -f memory.dmp --profile=Win7SP1x64 filescan | grep flag
volatility -f memory.dmp --profile=Win7SP1x64 dumpfiles -Q 0xADDR -D out/
volatility -f memory.dmp --profile=Win7SP1x64 cmdscan
volatility -f memory.dmp --profile=Win7SP1x64 memdump -p PID -D out/
volatility -f memory.dmp --profile=Win7SP1x64 hashdump # NTLM hashes
volatility -f memory.dmp --profile=Win7SP1x64 screenshot # desktop screenshot
volatility -f memory.dmp --profile=Win7SP1x64 clipboard # clipboard contents
```

TIP: In CTF memory dumps: always run pslist, cmdline, filescan, and clipboard first. Flags are often in clipboard or command history.

8. Disk Image Forensics

Autopsy & FTK Imager

```
# Autopsy (GUI – best for beginners)
# 1. Open Autopsy -> New Case
# 2. Add disk image (.img, .dd, .E01)
# 3. Run Ingest Modules: Keyword Search, File Type ID
# 4. Check: Deleted Files, File Views -> By Type
# 5. Search keyword 'flag' in Keyword Search

# Mount disk image to browse files
sudo mount -o loop disk.img /mnt/disk
ls -la /mnt/disk
grep -r 'flag' /mnt/disk/ 2>/dev/null
sudo umount /mnt/disk
```

Command Line Disk Analysis

```
# List partitions in disk image
mmls disk.img
fdisk -l disk.img

# Mount specific partition (offset from mmls output * 512)
sudo mount -o loop,offset=1048576 disk.img /mnt/part

# Recover deleted files
foremost -i disk.img -o recovered/
photorec disk.img # GUI-based recovery
scalpel disk.img -o out/

# Find all files (including deleted) with The Sleuth Kit
fls -r disk.img
fls -r disk.img | grep -i flag

# Extract specific inode
icat disk.img INODE_NUMBER > recovered_file

# Strings across entire disk
strings disk.img | grep -i flag
```

9. Document & Misc Forensics

PDF Analysis

```
# Extract text from PDF
pdftotext document.pdf output.txt
cat output.txt | grep -i flag

# PDF structure analysis
pdfid document.pdf          # check for JavaScript, embedded files
pdf-parser document.pdf     # parse all PDF objects

# Extract embedded files from PDF
pdfextract document.pdf
binwalk -e document.pdf

# JavaScript in PDF (could execute/decode)
pdfid document.pdf | grep JS
pdf-parser --search javascript document.pdf

# Strings
strings document.pdf | grep -i flag
```

Steganography in Other File Types

```
# Text file whitespace stego (spaces/tabs encode data)
cat -A file.txt          # show special chars (^I = tab, $ = newline)
# Tool: snow (whitespace stego)
snow -C file.txt         # decode
snow -C -p password file.txt

# Zero-width character stego (looks blank but has hidden Unicode)
# Copy text to:330k.github.io/stegcloak (decode)
# Or: Unicode steganography detector online

# Twitter/social media text stego
# Check for zero-width joiners \u200d, \u200b in text
python3 -c "text=open('file.txt').read(); print([hex(ord(c)) for c in text if ord(c)>127])"

# Binary file: check for appended data after EOF marker
# Example: JPEG ends at FF D9, anything after is appended data
xxd image.jpg | tail -20  # check end of file
```

10. Complete Tools Reference

Tool	Use For	Key Command / URL
file	Identify file type	file suspicious_file
exiftool	All metadata	exiftool file.jpg
strings	Readable text in binary	strings file grep -i flag
binwalk	Embedded files	binwalk -e file
foremost	File carving	foremost -i disk.img -o out/
xxd / hexdump	Hex view	xxd file head
zsteg	PNG LSB stego	zsteg -a image.png
steghide	JPG/BMP stego	steghide extract -sf file.jpg
stegseek	Crack steghide password	stegseek file.jpg rockyou.txt
Audacity	Audio spectrogram	Open -> Spectrogram view
Wireshark	PCAP analysis	Open file -> Follow TCP Stream

tshark	CLI PCAP analysis	<code>tshark -r file.pcap -Y http</code>
volatility3	Memory forensics	<code>vol3 -f mem.dmp windows.pslist</code>
Autopsy	Disk forensics GUI	GUI tool
fcrackzip	Crack ZIP password	<code>fcrackzip -u -D -p rockyou.txt a.zip</code>
aperisolve.com	Online all-in-one stego	Upload image -> runs all tools
stegonline.georgeom.net	Online bit plane viewer	Browser – view bit planes
hexed.it	Online hex editor	Browser – view/edit hex
georgeom.net/StegOnline	Online stego analysis	Bit planes, colour channels
29a.ch/sandbox	Online file analysis	Browser – analyze suspicious files

11. Master Forensics Checklist

Run through this for any forensics challenge — tick off each step:

#	Command	What you're looking for
1	<code>file <filename></code>	Real file type — may differ from extension
2	<code>exiftool <filename></code>	Flag in Comment, Author, GPS, Description
3	<code>strings <file> grep -i flag</code>	Plaintext flag embedded in binary
4	<code>binwalk <file></code>	Embedded ZIPs, PNGs, ELFs inside file
5	<code>binwalk -e <file></code>	Extract all embedded files
6	<code>xxd <file> head -20</code>	Magic bytes — verify and fix file type
7	<code>zsteg -a <file.png></code>	LSB steganography in PNG
8	<code>steghide extract -sf <file></code>	Data hidden in JPG/BMP
9	<code>stegseek <file> rockyou.txt</code>	Brute-force steghide password
10	Audacity spectrogram	Flag visible in frequency domain of audio
11	Wireshark -> Follow TCP Stream	Credentials, files in network traffic
12	File -> Export Objects -> HTTP	Files transferred in PCAP
13	<code>vol3 -f mem.dmp windows.pslist</code>	Suspicious processes in memory dump
14	<code>vol3 windows.cmdline / consoles</code>	Typed commands, flags in memory
15	<code>vol3 windows.filescan grep flag</code>	Flag files in memory
16	<code>foremost -i disk.img -o out/</code>	Recover deleted files from disk
17	<code>aperisolve.com (online)</code>	Runs all stego tools at once on image